

Interlude: Iteration Without Materialization

Opening Question

Does a program need to construct a new collection before it can describe what should happen to every element of an existing one?

1. Repetition over a collection

Begin with the ordinary case.

```
let numbers = vec![1, 2, 3];

for number in &numbers {
    println!("{number}");
}
```

Writing `for number in &numbers` borrows `numbers` for the duration of the loop, produces one `&i32` per element in turn, and leaves `numbers` itself fully intact and reachable afterward. Nothing here has been consumed or transformed yet; the loop only observes.

2. Three ways to enter iteration

That borrowing choice is one of three, and Rust names all three explicitly rather than leaving the choice implicit.

```
for n in numbers.iter() { /* &i32 - observe */ }
for n in numbers.iter_mut() { /* &mut i32 -authorized to mutate */ }
for n in numbers.into_iter() { /* i32 -consumed and transferred */ }
```

`.iter()` grants knowledge of each element without authority over it. `.iter_mut()` grants a temporary authorization to mutate each element in place, without transferring ownership of the collection itself. `.into_iter()` consumes the collection outright, transferring ownership of each element out of it one at a time; after this loop, `numbers` no longer exists. These three are not stylistic variants of one operation. They are the same three-way distinction, observation versus mutation-in-place versus consuming transfer, that shows up wherever Rust grants access to a value at all, applied here to what a loop is permitted to do with each element in turn.

3. The iterator as suspended production

An iterator is not the collection, and it is not a second collection built to hold the results. It is a rule for producing the next item, together with whatever state is needed to know what “next” means the next time it is asked. Calling `.iter()` on a `Vec` does not walk the vector; it constructs a small value, an iterator, that has not yet produced anything and will not until something asks it to.

4. `next` makes continuation explicit

That asking has an exact name. Chapter 10 showed a type implementing `Iterator` by defining `next`; here, the same method is worth calling directly, from the consuming side, to see what it actually hands back.

```
let mut it = numbers.iter();
println!("{:?}", it.next()); // Some(1)
println!("{:?}", it.next()); // Some(2)
println!("{:?}", it.next()); // Some(3)
println!("{:?}", it.next()); // None
```

Each call either advances production and returns `Some(item)`, or reports that the sequence has ended by returning `None`. `Option<Item>` is doing exactly the job Chapter 13 gives it elsewhere: recording, honestly, that “no further item” is a distinct and expected outcome rather than a special sentinel value smuggled into the item type itself. Every other iterator method, however elaborate, is ultimately just a disciplined way of calling `next` repeatedly on the consumer’s behalf.

5. Adapters reshape future production

An adapter takes an iterator and returns a new iterator describing a changed production rule, without running anything itself.

```
let evens_doubled = numbers
    .iter()
    .filter(|&n| n % 2 == 0)
    .map(|&n| n * 2);
```

`.filter(...)` describes a rule that skips whatever the inner iterator produces until an item passes the predicate. `.map(...)` describes a rule that transforms whatever the inner iterator hands it. Neither call has, at this point, looked at a single element of `numbers`.

6. Laziness

That last claim is worth demonstrating rather than asserting, since it's easy to read the chain above and assume the transformation already happened.

```
let chain = numbers.iter().map(|n| {
    println!("touching {n}");
    n * 2
});
println!("chain built, nothing printed yet");
let doubled: Vec<i32> = chain.collect();
// only now does "touching 1" / "touching 2" / "touching 3" appear
```

Building `chain` does not run the closure passed to `map` even once. The chain is a composed *description* of a possible computation, in exactly the sense Chapter 2's composition argument already established for adapter chains in general; what this example adds is visible proof that the description and its execution are two separate events, separated here by an explicit `println!` that only fires once something finally drives the iterator forward.

7. Consumers create consequences

That driving-forward has its own name too: a consumer. `sum`, `count`, `for_each`, `find`, and `collect` are all consumers — each one repeatedly calls `next` on whatever chain precedes it until the chain reports `None`, and produces a final value or effect from what it saw along the way. Nothing before a consumer in a chain has done any work; everything after one is finished.

8. Why collect needs a destination

Most consumers have an unambiguous result type: `sum` produces a number, `count` produces a `usize`. `collect` is different, because many different collection types could equally well receive the same sequence of items.

```
let doubled: Vec<i32> = numbers.iter().map(|n| n * 2).collect();
// or, without a type annotation on the binding:
let doubled = numbers.iter().map(|n| n * 2).collect::<Vec<i32>>();
```

The iterator chain itself carries no information about whether its output should become a `Vec`, a `HashSet`, or a `String`; `collect` is generic over its return type, and something else, a type annotation on the binding or the explicit `::<Vec<i32>>` turbofish syntax, has to supply the missing piece. This is not a quirk of `collect` specifically. It is the same parametric-admissibility discipline Chapter 10 formalized: `collect`'s bound tells the compiler how to build *any* sufficiently well-behaved destination, and the caller still has to say which one is meant.

9. Short-circuiting and narrowed search

Because production only happens on demand, a consumer need not exhaust the whole chain to produce its answer.

```
let first_even = numbers.iter().find(|&n| n % 2 == 0);
```

`find` calls `next` only until a matching item appears, then stops; if `numbers` held ten million elements and the first one already matched, the other 9,999,999 would never be produced at all. `any`, and a bounded `take` followed by any consumer, work the same way. Laziness is not merely a description of when work happens; here it directly reduces how much work happens, by letting the consumer decide when enough of the sequence has been seen.

10. Loops and chains are not enemies

The chain from Section 5 and an ordinary loop describe the same transformation at two levels of compression.

```
// as a chain
let result: Vec<i32> = numbers.iter()
    .filter(|&n| n % 2 == 0)
    .map(|&n| n * 2)
```

```

        .collect();

// as a loop
let mut result = Vec::new();
for &n in numbers.iter() {
    if n % 2 == 0 {
        result.push(n * 2);
    }
}

```

Neither form is being ranked above the other here. The chain names each stage as an inspectable value and makes the description-versus-execution split of Section 6 visible in the syntax itself; the loop makes the same transformation explicit as a sequence of statements. Translating between them is a useful exercise precisely because it shows the identical continuation being carried out by two different means, one compressed, one expanded.

Invariant Established

An iterator separates the description of a traversal from its materialized consequence. Its adapters reshape what may be produced next without producing anything themselves; its consumers decide when that latent traversal becomes actual, and how much of it needs to run at all.

Non-overlap note: Chapter 2 supplies the composition theory behind adapter chains, including the `Fn`/`FnMut`/`FnOnce` treatment of closures, and Chapter 10 supplies the generic-trait theory behind `Iterator`'s one required method and the functor law for `map`. This interlude does not re-argue either. It supplies the operational syntax and execution model those two chapters assume but do not separately walk through: `iter`/`iter_mut`/`into_iter`, `next` called from the consuming side, laziness demonstrated rather than asserted, `collect`'s destination-type problem, and short-circuiting.